7.9 Review Questions and Exercises (복습 문제 및 연습 문제)

7.9.1 Short Answer (단답형)

1. In the following code sequence, show the value of AL after each shift or rotate instruction has executed:

```
mov al,0D4h
shr al,1          ; a.
mov al,0D4h
sar al,1          ; b.
mov al,0D4h
sar al,4          ; c.
mov al,0D4h
rol al,1          ; d.
```

1. 다음 코드 시퀀스에서, 각 시프트 또는 회전(rotate) 명령이 실행된 후의 AL 값을 보이시오.

2. In the following code sequence, show the value of AL after each shift or rotate instruction has executed:

```
mov al,0D4h
ror al,3          ; a.
mov al,0D4h
rol al,7          ; b.
stc
mov al,0D4h
rcl al,1          ; c.
stc
mov al,0D4h
rcr al,3          ; d.
```

2. 다음 코드 시퀀스에서, 각 시프트 또는 회전 명령이 실행된 후의 AL 값을 보이시오.

3. What will be the contents of AX and DX after the following operation?

```
mov dx,0
mov ax,222h
mov cx,100h
mul cx
```

3. 다음 연산 후 AX와 DX의 내용은 무엇인가?

4. What will be the contents of AX after the following operation?

```
mov ax,63h
mov bl,10h
div bl
```

4. 다음 연산 후 AX의 내용은 무엇인가?

5. What will be the contents of EAX and EDX after the following operation?

```
mov eax,123400h
mov edx,0
mov ebx,10h
div ebx
```

5. 다음 연산 후 EAX와 EDX의 내용은 무엇인가?

6. What will be the contents of AX and DX after the following operation?

```
mov ax,4000h
mov dx,500h
mov bx,10h
div bx
```

6. 다음 연산 후 AX와 DX의 내용은 무엇인가?

7. What will be the contents of BX after the following instructions execute?

```
mov bx,5
stc
mov ax,60h
adc bx,ax
```

7. 다음 명령들이 실행된 후 BX의 내용은 무엇인가?

8. Describe the output when the following code executes in 64-bit mode:

```
.data
dividend_hi QWORD 00000108h
dividend_lo QWORD 33300020h
divisor     QWORD 00000100h
.code
mov rdx,dividend_hi
mov rax,dividend_lo
div divisor
```

↓ 나눌 overflow 발생, 출력 X

8. 다음 코드가 64비트 모드에서 실행될 때의 출력(결과)을 설명하시오.

9. The following program is supposed to subtract val2 from val1. Find and correct all logic errors (CLC clears the Carry flag):

```
.data
val1 QWORD 20403004362047A1h
val2 QWORD 055210304A2630B2h
result QWORD 0

.code
     mov  cx,8             ; loop counter
     mov  esi,val1         ; set index to start
     mov  edi,val2
     clc                   ; clear Carry flag
top:
     mov  al,BYTE PTR[esi]   ; get first number
     sbb  al,BYTE PTR[edi]   ; subtract second
     mov  BYTE PTR[esi],al   ; store the result
     dec  esi
     dec  edi
     loop top
```

1. esi, edi 에 값이 아니라 주소가 들어가야하므로 offset val1, offset val2 넣어야함
2. 32bit 100번 ECX로 쓰므로 mov ecx, 4 이어야함
3. 둘 ecx가 아닌 inc
4. 결과로 result 저장해야하는데 val1 은 덮어씀

9. 다음 프로그램은 val1에서 val2를 빼도록 되어 있다. 모든 논리 오류를 찾아 수정하시오. (CLC는 캐리 플래그를 클리어한다.)

10. What will be the hexadecimal contents of RAX after the following instructions execute in 64-bit mode?

```
.data
multiplicand QWORD 0001020304050000h
.code
imul rax,multiplicand, 4
```

RAX = 0004080C1014000h

10. 다음 명령들이 64비트 모드에서 실행된 후 RAX의 16진수 내용은 무엇인가?

7.9.2 Algorithm Workbench (알고리즘 워크벤치)

1. Write a sequence of shift instructions that cause AX to be sign-extended into EAX. In other words, the sign bit of AX is copied into the upper 16 bits of EAX. Do not use the CWD instruction.

1. AX가 EAX로 부호 확장(sign-extend)되도록 하는 시프트 명령들의 시퀀스를 작성하시오. 즉, AX의 부호 비트가 EAX의 상위 16비트에 복사되어야 한다. CWD 명령은 사용하지 마시오.

```
shl eax,16
sar eax,16
```

2. Suppose the instruction set contained no rotate instructions. Show how you would use SHR and a conditional jump instruction to rotate the contents of the AL register 1 bit to the right.

2. 명령어 집합에 회전(rotate) 명령이 없다고 가정하자. SHR과 조건부 점프 명령을 사용하여 AL 레지스터의 내용을 오른쪽으로 1비트 회전시키는 방법을 보이시오.

```
shr al,1
jnc done
or al,80h
done:
```

3. Write a logical shift instruction that multiplies the contents of EAX by 16.

3. EAX의 내용에 16을 곱하는 논리 시프트 명령을 작성하시오.

```
shl eax,4
```

4. Write a logical shift instruction that divides EBX by 4.

4. EBX를 4로 나누는 논리 시프트 명령을 작성하시오.

```
shr ebx,2
```

5. Write a single rotate instruction that exchanges the high and low halves of the DL register.

5. DL 레지스터의 상위 절반과 하위 절반을 서로 교환하는 단일 회전 명령을 작성하시오.

```
rol dl,4
```

6. Write a single SHLD instruction that shifts the highest bit of the AX register into the lowest bit position of DX and shifts DX one bit to the left.

```
shld dx,ax,1
```

6. AX 레지스터의 최상위 비트를 DX의 최하위 비트 위치로 시프트하고, DX를 왼쪽으로 1비트 시프트하는 단일 SHLD 명령을 작성하시오.

7. Write a sequence of instructions that shift three memory bytes to the right by 1 bit position. Use the following test data:

```
byteArray BYTE 81h,20h,33h
```

```
shr byteArray+2,1
rcr byteArray+1,1
rcr byteArray,1
```

7. 메모리상의 3바이트를 오른쪽으로 1비트 위치만큼 시프트하는 명령들의 시퀀스를 작성하시오. 다음 테스트 데이터를 사용하시오.

8. Write a sequence of instructions that shift three memory words to the left by 1 bit position. Use the following test data:

```
wordArray WORD 810Dh, 0C064h,93ABh
```

8. 메모리상의 3개의 워드(word)를 왼쪽으로 1비트 위치만큼 시프트하는 명령들의 시퀀스를 작성하시오. 다음 테스트 데이터를 사용하시오.

```
shl wordArray,1
rcl wordArray+1,1
rcl wordArray+2,1
```

9. Write instructions that multiply −5 by 3 and store the result in a 16-bit variable val1.

9. −5에 3을 곱하고 그 결과를 16비트 변수 val1에 저장하는 명령들을 작성하시오.

10. Write instructions that divide −276 by 10 and store the result in a 16-bit variable val1.

10. −276을 10으로 나누고 그 결과를 16비트 변수 val1에 저장하는 명령들을 작성하시오.

11. Implement the following C++ expression in assembly language, using 32-bit unsigned operands:

```
val1 = (val2 * val3) / (val4 - 3)
```

11. 다음 C++ 식을 32비트 부호 없는(unsigned) 피연산자를 사용하여 어셈블리어로 구현하시오.

12. Implement the following C++ expression in assembly language, using 32-bit signed operands:

```
val1 = (val2 / val3) * (val1 + val2)
```

12. 다음 C++ 식을 32비트 부호 있는(signed) 피연산자를 사용하여 어셈블리어로 구현하시오.

13. Write a procedure that displays an unsigned 8-bit binary value in decimal format. Pass the binary value in AL. The input range is limited to 0 to 99, decimal. The only procedure you can call from the book's link library is WriteChar. The procedure should contain approximately eight instructions. Here is a sample call:

```
mov  al,65           ; range limit: 0 to 99
call showDecimal8
```

13. 부호 없는 8비트 이진 값을 10진수 형식으로 출력하는 프로시저를 작성하시오. 이진 값은 AL에 전달한다. 입력 범위는 10진수로 0에서 99까지로 제한된다. 이 책의 링크 라이브러리에서 호출할 수 있는 프로시저는 WriteChar뿐이다. 프로시저는 대략 8개의 명령으로 구성되어야 한다. 다음은 호출 예시이다.

14. Challenge: Suppose AX contains 0072h and the Auxiliary Carry flag is set as a result of adding two unknown ASCII decimal digits. Use the Intel 64 and IA-32 Instruction Set Reference to determine what output the AAA instruction would produce. Explain your answer.

14. 도전 과제: AX에 0072h가 들어 있고, 알 수 없는 두 ASCII 10진수 숫자를 더한 결과로 보조 캐리 플래그(Auxiliary Carry flag)가 설정되었다고 가정하자. Intel 64 and IA-32 Instruction Set Reference를 사용하여 AAA 명령이 어떤 결과를 만들어 내는지 알아내고, 답을 설명하시오.

15. Challenge: Using only SUB, MOV, and AND instructions, show how to calculate x = n mod y, assuming that you are given the values of n and y. You can assume that n is any 32-bit unsigned integer, and y is a power of 2.

15. 도전 과제: SUB, MOV, AND 명령만을 사용하여 x = n mod y를 계산하는 방법을 보이시오. n과 y의 값이 주어진다고 가정한다. n은 임의의 32비트 부호 없는 정수이고, y는 2의 거듭제곱이라고 가정해도 된다.

16. Challenge: Using only SAR, ADD, and XOR instructions (but no conditional jumps), write code that calculates the absolute value of the signed integer in the EAX register. Hints: A number can be negated by adding −1 to it and then forming its one's complement. Also, if you XOR an integer with all 1s, its 1s are reversed. On the other hand, if you XOR an integer with all zeros, the integer is unchanged.

16. 도전 과제: SAR, ADD, XOR 명령만을 사용하고 (조건부 점프는 사용하지 않고), EAX 레지스터에 있는 부호 있는 정수의 절댓값을 계산하는 코드를 작성하시오. 힌트: 어떤 수에 −1을 더한 다음 1의 보수를 취하면 그 수를 부정(negate)할 수 있다. 또한 정수를 모두 1인 값과 XOR하면 그 정수의 1들이 반전된다. 반대로, 정수를 모두 0인 값과 XOR하면 그 정수는 변하지 않는다.

7.10 Programming Exercises (프로그래밍 연습 문제)

★ 1. Display ASCII Decimal
Write a procedure named WriteScaled that outputs a decimal ASCII number with an implied decimal point. Suppose the following number were defined as follows, where DECIMAL_OFFSET indicates that the decimal point must be inserted five positions from the right side of the number:

```
DECIMAL_OFFSET = 5
.data
decimal_one BYTE "100123456789765"
```

★ 1. ASCII 10진수 출력
암시된 소수점을 가진 10진수 ASCII 숫자를 출력하는 WriteScaled라는 이름의 프로시저를 작성하시오. 다음과 같이 숫자가 정의되어 있다고 가정하자. 여기서 DECIMAL_OFFSET은 소수점을 숫자의 오른쪽 끝에서 다섯 번째 위치에 삽입해야 함을 나타낸다.

WriteScaled would display the number like this:

WriteScaled는 이 숫자를 다음과 같이 표시할 것이다.

```
1001234567.89765
```

When calling WriteScaled, pass the number's offset in EDX, the number length in ECX, and the decimal offset in EBX. Write a test program that passes three numbers of different sizes to the WriteScaled procedure.

WriteScaled를 호출할 때, 숫자의 오프셋은 EDX에, 숫자의 길이는 ECX에, 소수점 오프셋은 EBX에 전달한다. 서로 다른 크기의 숫자 세 개를 WriteScaled 프로시저에 전달하는 테스트 프로그램을 작성하시오.

★★ 2. Extended Subtraction Procedure
Create a procedure named Extended_Sub that subtracts two binary integers of arbitrary size. The storage size of the two integers must be the same, and their size must be a multiple of 32 bits. Write a test program that passes several pairs of integers, each at least 10 bytes long.

★★ 2. 확장 뺄셈 프로시저
임의의 크기를 가진 두 이진 정수를 빼는 Extended_Sub라는 이름의 프로시저를 작성하시오. 두 정수의 저장 크기는 같아야 하며, 그 크기는 32비트의 배수여야 한다. 각각 최소 10바이트 길이인 정수 쌍 여러 개를 전달하는 테스트 프로그램을 작성하시오.

★★ 3. Packed Decimal Conversion
Write a procedure named PackedToAsc that converts a 4-byte packed decimal integer to a string of ASCII decimal digits. Pass the packed integer and the address of a buffer holding the ASCII digits to the procedure. Write a short test program that passes at least 5 packed decimal integers to your procedure.

★★ 3. 패킹된 10진수 변환
4바이트 패킹된 10진수(packed decimal) 정수를 ASCII 10진수 숫자 문자열로 변환하는 PackedToAsc라는 이름의 프로시저를 작성하시오. 패킹된 정수와 ASCII 숫자를 담을 버퍼의 주소를 프로시저에 전달한다. 최소 5개의 패킹된 10진수 정수를 프로시저에 전달하는 짧은 테스트 프로그램을 작성하시오.

★★ 4. Encryption Using Rotate Operations
Write a procedure that performs simple encryption by rotating each plaintext byte a varying number of positions in different directions. For example, in the following array that represents the encryption key, a negative value indicates a rotation to the left and a positive value indicates a rotation to the right. The integer in each position indicates the magnitude of the rotation:

```
key BYTE -2, 4, 1, 0, -3, 5, 2, -4, -4, 6
```

★★ 4. 회전 연산을 이용한 암호화
각 평문 바이트를 서로 다른 방향으로 가변적인 위치 수만큼 회전시켜 간단한 암호화를 수행하는 프로시저를 작성하시오. 예를 들어, 암호화 키를 나타내는 다음 배열에서 음수 값은 왼쪽 회전을, 양수 값은 오른쪽 회전을 나타낸다. 각 위치의 정수는 회전의 크기를 나타낸다.

Your procedure should loop through a plaintext message and align the key to the first 10 bytes of the message. Rotate each plaintext byte by the amount indicated by its matching key array value. Then, align the key to the next 10 bytes of the message and repeat the process. Write a program that tests your encryption procedure by calling it twice, with different data sets.

프로시저는 평문 메시지를 반복 처리하면서 키를 메시지의 처음 10바이트에 맞춰 정렬해야 한다. 각 평문 바이트를 이에 대응하는 키 배열 값이 나타내는 만큼 회전시킨다. 그런 다음 키를 메시지의 다음 10바이트에 맞춰 정렬하고 이 과정을 반복한다. 서로 다른 데이터 집합으로 암호화 프로시저를 두 번 호출하여 테스트하는 프로그램을 작성하시오.

★★★ 5. Prime Numbers
Write a program that generates all prime numbers between 2 and 1000, using the Sieve of Eratosthenes method. You can find many articles that describe the method for finding primes in this manner on the Internet. Display all the prime values.

★★★ 5. 소수
에라토스테네스의 체(Sieve of Eratosthenes) 방법을 사용하여 2에서 1000 사이의 모든 소수를 생성하는 프로그램을 작성하시오. 인터넷에서 이러한 방식으로 소수를 찾는 방법을 설명하는 많은 글을 찾을 수 있다. 모든 소수 값을 출력하시오.

★★★ 6. Greatest Common Divisor (GCD)
The greatest common divisor (GCD) of two integers is the largest integer that will evenly divide both integers. The GCD algorithm involves integer division in a loop, described by the following pseudocode:

```
int GCD(int x, int y)
{
    x = abs(x)              // absolute value
    y = abs(y)
    do {
        int n = x % y
        x = y
        y = n
    } while (y > 0)
    return x
}
```

★★★ 6. 최대공약수 (GCD)
두 정수의 최대공약수(GCD)는 두 정수를 모두 나누어떨어지게 하는 가장 큰 정수이다. GCD 알고리즘은 반복문 안에서 정수 나눗셈을 수행하며, 다음 의사코드(pseudocode)로 설명된다.

Implement this function in assembly language and write a test program that calls the function several times, passing it different values. Display all results on the screen.

이 함수를 어셈블리어로 구현하고, 서로 다른 값을 전달하며 이 함수를 여러 번 호출하는 테스트 프로그램을 작성하시오. 모든 결과를 화면에 출력하시오.

★★★ 7. Bitwise Multiplication
Write a procedure named BitwiseMultiply that multiplies any unsigned 32-bit integer by EAX, using only shifting and addition. Pass the integer to the procedure in the EBX register, and return the product in the EAX register. Write a short test program that calls the procedure and displays the product. (We will assume that the product is never larger than 32 bits.) This is a fairly challenging program to write. One possible approach is to use a loop to shift the multiplier to the right, keeping track of the number of shifts that occur before the Carry flag is set. The resulting shift count can then be applied to the SHL instruction, using the multiplicand as the destination operand. Then, the same process must be repeated until you find the last 1 bit in the multiplier.

★★★ 7. 비트 단위 곱셈
시프트와 덧셈만을 사용하여 임의의 부호 없는 32비트 정수에 EAX를 곱하는 BitwiseMultiply라는 이름의 프로시저를 작성하시오. 정수는 EBX 레지스터로 프로시저에 전달하고, 곱은 EAX 레지스터로 반환한다. 이 프로시저를 호출하고 곱을 출력하는 짧은 테스트 프로그램을 작성하시오. (곱은 32비트를 넘지 않는다고 가정한다.) 이것은 작성하기가 상당히 어려운 프로그램이다. 한 가지 가능한 접근법은 반복문을 사용하여 승수(multiplier)를 오른쪽으로 시프트하면서, 캐리 플래그가 설정되기 전까지 발생한 시프트 횟수를 추적하는 것이다. 그렇게 얻은 시프트 횟수를, 피승수(multiplicand)를 목적지 피연산자로 하여 SHL 명령에 적용할 수 있다. 그런 다음 승수의 마지막 1 비트를 찾을 때까지 같은 과정을 반복해야 한다.

★★★ 8. Add Packed Integers
Extend the AddPacked procedure from Section 7.6.1 so that it adds two packed decimal integers of arbitrary size (both lengths must be the same). Write a test program that passes AddPacked several pairs of integers: 4-byte, 8-byte, and 16-byte. We suggest that you use the following registers to pass information to the procedure:

```
ESI - pointer to the first number
EDI - pointer to the second number
EDX - pointer to the sum
ECX - number of bytes to add
```

★★★ 8. 패킹된 정수 더하기
7.6.1절의 AddPacked 프로시저를 확장하여, 임의의 크기를 가진 두 패킹된 10진수 정수를 더할 수 있도록 하시오. (두 길이는 반드시 같아야 한다.) AddPacked에 4바이트, 8바이트, 16바이트 정수 쌍을 여러 개 전달하는 테스트 프로그램을 작성하시오. 프로시저에 정보를 전달하기 위해 다음 레지스터를 사용할 것을 권장한다.